Universidad Autónoma de Madrid
Escuela Politécnica Superior

Aprendizaje Automático III

Proyecto de Generative AI

Grado en Ciencia e Ingeniería de Datos

Autores:
Sara Echeverribar Lalaguna
Raquel Vallejo Rincón

Tutor:
Alberto Suárez González

Abril 2026

Repositorio del proyecto

El código fuente desarrollado, junto con los notebooks, scripts auxiliares y recursos generados durante el proyecto, se encuentra disponible en el siguiente repositorio de GitHub:

`https://github.com/saraeche14-bot/generative-ai-diffusion-models`

Debido al tamaño de algunos archivos, especialmente los checkpoints de los modelos entrenados, estos pueden no estar incluidos directamente en la entrega principal. El repositorio permite consultar la estructura completa del proyecto, la implementación de los modelos, los experimentos realizados y los materiales necesarios para reproducir los resultados principales.

Resumen

En los últimos años, la inteligencia artificial generativa ha experimentado un gran avance, especialmente en la generación de imágenes. Entre las distintas aproximaciones, los modelos de difusión han destacado por su estabilidad en el entrenamiento y su capacidad para generar muestras de alta calidad.

Este proyecto se centra en el estudio, implementación y evaluación de modelos generativos basados en difusión. En particular, se han considerado dos enfoques principales: modelos de Variance Exploding basados en movimiento browniano y modelos de Variance Preserving basados en el proceso de Ornstein-Uhlenbeck. Asimismo, se han analizado diferentes planificaciones de ruido y métodos de muestreo para evaluar su impacto en la calidad de las imágenes generadas.

Los modelos se han entrenado sobre el dataset MNIST, incluyendo una versión extendida con información de color para permitir la generación condicionada. Esto permite controlar tanto el dígito como el color de las muestras generadas. Además, se ha explorado una tarea de reconstrucción de imágenes incompletas (imputation) para evaluar la capacidad del modelo de inferir información faltante.

El rendimiento de los modelos se ha evaluado mediante métricas cuantitativas como el Fréchet Inception Distance (FID), Bits per Dimension (BPD) y el Inception Score (IS), junto con análisis cualitativos. Los resultados muestran que el modelo basado en Ornstein-Uhlenbeck con planificación de ruido tipo coseno ofrece el mejor rendimiento, obteniendo menores valores de FID y generando imágenes más coherentes.

En conjunto, los experimentos confirman la eficacia de los modelos de difusión para tareas de generación de imágenes, así como su flexibilidad para problemas de generación condicionada y reconstrucción.

Palabras clave: modelos de difusión, modelos generativos, generación de imágenes, score-based models, MNIST, generación condicionada

Glosario

Score: Gradiente del logaritmo de la densidad de probabilidad, $\nabla_x \log p_t(x)$.

Forward process: Proceso de difusión que transforma progresivamente los datos originales en ruido.

Reverse process: Proceso inverso que parte de ruido y genera muestras similares a los datos originales.

Noise schedule: Función que controla la intensidad del ruido añadido a lo largo del tiempo.

Sampler: Método numérico utilizado para simular el proceso inverso de generación.

Checkpoint: Archivo donde se almacenan los pesos entrenados de un modelo.

Imputation: Tarea de reconstrucción de regiones faltantes de una imagen a partir de la información observada.

Embedding: Representación vectorial utilizada para incorporar información adicional al modelo, como el tiempo, el dígito o el color.

Acrónimos

SDE: Stochastic Differential Equation.
ODE: Ordinary Differential Equation.
VE: Variance Exploding.
VP: Variance Preserving.
BM: Brownian Motion.
OU: Ornstein-Uhlenbeck.
BPD: Bits per Dimension.
FID: Fréchet Inception Distance.
IS: Inception Score.
MNIST: Modified National Institute of Standards and Technology.

Índice general

Repositorio del proyecto III

1. Introducción 1
1.1. Motivación . . . 1
1.2. Objetivos del proyecto . . . 1
1.3. Estructura del documento . . . 1

2. Estado del arte 2
2.1. Modelos generativos . . . 2
2.2. Modelos basados en difusión . . . 2
2.3. Procesos estocásticos y ecuaciones diferenciales . . . 2
2.4. Modelos generativos basados en score . . . 2
2.5. Procesos de difusión utilizados . . . 3
2.5.1. Movimiento Browniano (Variance Exploding) . . . 3
2.5.2. Proceso de Ornstein-Uhlenbeck (Variance Preserving) . . . 3
2.6. Planificaciones de ruido . . . 3
2.7. Métodos de generación . . . 3
2.7.1. Integrador Euler-Maruyama . . . 3
2.7.2. Probability Flow ODE . . . 4
2.7.3. Método Predictor-Corrector . . . 4
2.8. Métricas de evaluación para modelos generativos . . . 4
2.8.1. Negative Log-Likelihood . . . 4
2.8.2. Bits per Dimension . . . 4
2.8.3. Fréchet Inception Distance . . . 4
2.8.4. Inception Score . . . 4

3. Diseño del sistema 5
3.1. Descripción general del sistema . . . 5
3.2. Análisis de requisitos . . . 5
3.2.1. Requisitos funcionales . . . 5
3.2.2. Requisitos no funcionales . . . 6
3.3. Casos de uso . . . 6
3.3.1. Generación de imágenes a partir de ruido . . . 6
3.3.2. Generación condicionada por dígito y color . . . 7
3.3.3. Evaluación de modelos generativos . . . 7
3.3.4. Reconstrucción de imágenes incompletas (Imputation) . . . 7
3.4. Arquitectura del modelo . . . 7
3.4.1. Modelo ScoreNet basado en U-Net . . . 7
3.4.2. Diagrama de clases del sistema . . . 7
3.4.3. Codificación temporal mediante Fourier features . . . 8
3.5. Validación y pruebas . . . 9
3.6. Proceso de entrenamiento . . . 9
3.7. Proceso de generación de imágenes . . . 9
3.8. Generación condicionada . . . 10
3.9. Organización del código . . . 10
3.10. Gestión de modelos entrenados y checkpoints . . . 10

4. Desarrollo e implementación **11**
4.1. Metodología de desarrollo . . . 11
4.2. Planificación del proyecto . . . 11
4.3. Preparación de los datos . . . 11
4.3.1. MNIST en escala de grises . . . 11
4.3.2. MNIST coloreado . . . 12
4.4. Implementación de los procesos de difusión . . . 12
4.4.1. Forward process . . . 12
4.4.2. Reverse process . . . 12
4.5. Entrenamiento de los modelos . . . 13
4.5.1. Modelo Brownian Motion . . . 13
4.5.2. Modelo Ornstein-Uhlenbeck . . . 13
4.5.3. Modelo condicionado para generación controlada . . . 13
4.6. Implementación de los métodos de muestreo . . . 13
4.7. Validación del funcionamiento . . . 13
4.8. Entorno de desarrollo y recursos computacionales . . . 14
4.9. Consideraciones éticas y sociales . . . 14

5. Resultados **15**
5.1. Resultados cualitativos . . . 15
5.1.1. Generación con Brownian Motion . . . 15
5.1.2. Generación con Ornstein-Uhlenbeck . . . 15
5.1.3. Generación condicionada . . . 15
5.2. Comparación de planificaciones de ruido . . . 16
5.3. Resultados cuantitativos . . . 16
5.3.1. Comparación de modelos y planificaciones de ruido . . . 16
5.3.2. Comparación de métodos de muestreo . . . 17
5.3.3. Evolución del modelo durante el entrenamiento . . . 17
5.4. Discusión de resultados . . . 18
5.5. Reconstrucción de imágenes (Imputation) . . . 18

6. Conclusiones **20**
6.1. Conclusiones principales . . . 20
6.2. Limitaciones del trabajo . . . 20
6.3. Líneas futuras . . . 21

Bibliografía **23**

A. Material adicional **25**
A.1. Derivación de las ecuaciones diferenciales estocásticas . . . 25
A.1.1. Ecuación de Fokker-Planck . . . 25
A.2. Proceso directo (forward diffusion) . . . 25
A.2.1. Caso Brownian Motion (Variance Exploding) . . . 25
A.2.2. Caso Ornstein-Uhlenbeck (Variance Preserving) . . . 26
A.3. Derivación del proceso inverso . . . 26
A.4. Estimación del score mediante redes neuronales . . . 27
A.4.1. Derivación de la función de pérdida . . . 27
A.5. Discretización: método de Euler-Maruyama . . . 27
A.6. Probability Flow ODE . . . 27
A.7. Interpretación intuitiva . . . 28

A.8. Resultados adicionales 28
A.8.1. Reconstrucción de imágenes (imputation) 28
A.8.2. Generación adicional 28
A.8.3. Resultados en CIFAR-10 29

Índice de tablas

5.1. Comparación de modelos, planificaciones de ruido y métricas de evaluación. 16
5.2. Comparación de métodos de muestreo y métricas de evaluación. 17
5.3. Evolución de las métricas FID, IS y BPD durante el entrenamiento. 17

Índice de figuras

3.1. Diagrama de clases del sistema de modelos de difusión implementado. . . . 8

4.1. Planificación temporal del desarrollo del proyecto. 11
4.2. Ejemplo del proceso forward de difusión, donde una imagen original se degrada progresivamente mediante la adición de ruido gaussiano. 12
4.3. Proceso inverso de difusión en el que una muestra inicial de ruido evoluciona hasta generar una imagen coherente mediante el modelo entrenado. 12

5.1. Ejemplos de imágenes generadas utilizando el modelo basado en movimiento browniano. 15
5.2. Ejemplos de imágenes generadas utilizando el modelo basado en Ornstein-Uhlenbeck. 15
5.3. Ejemplos de generación condicionada por dígito y color. 16
5.4. Evolución del FID durante el entrenamiento del modelo. 18
5.5. Ejemplo de reconstrucción de imágenes. En cada fila se muestra la imagen original (izquierda), la imagen reconstruida (centro) y la versión parcialmente observada (derecha). 19

A.1. Reconstrucción de imágenes con región central faltante. 28
A.2. Generación adicional con modelo OU. 28
A.3. Generación condicionada adicional. 29
A.4. Generación de imágenes de gatos en CIFAR-10. 29

Capítulo 1

Introducción

1.1. Motivación

La inteligencia artificial generativa ha experimentado un gran desarrollo en los últimos años, especialmente en la generación de imágenes. Modelos como GANs, VAEs y, más recientemente, los modelos de difusión, han demostrado una gran capacidad para aprender distribuciones complejas y generar muestras realistas.

Los modelos de difusión destacan por su estabilidad en el entrenamiento y la calidad de los resultados. Su funcionamiento se basa en la adición progresiva de ruido a los datos y el aprendizaje de su inversión, lo que permite un proceso de generación controlado y bien fundamentado desde el punto de vista probabilístico.

Este contexto motiva el estudio de estos modelos para comprender su comportamiento y evaluar su rendimiento en tareas de generación de imágenes.

1.2. Objetivos del proyecto

El objetivo de este proyecto es el estudio, implementación y evaluación de modelos de difusión. En particular, se plantean los siguientes objetivos:

- Implementar modelos basados en movimiento browniano (VE) y Ornstein-Uhlenbeck (VP).
- Analizar el efecto de distintas planificaciones de ruido.
- Evaluar métodos de muestreo para el proceso inverso.
- Desarrollar un modelo de generación condicionada por dígito y color.
- Comparar los resultados mediante FID e Inception Score.

1.3. Estructura del documento

El documento se organiza de la siguiente forma:

- Capítulo 2: fundamentos teóricos.
- Capítulo 3: diseño del sistema.
- Capítulo 4: implementación.
- Capítulo 5: resultados.
- Capítulo 6: conclusiones.

Capítulo 2

Estado del arte

2.1. Modelos generativos

Los modelos generativos tienen como objetivo aprender la distribución de probabilidad de un conjunto de datos para generar nuevas muestras sintéticas similares a las observadas. En el ámbito de generación de imágenes, estos modelos permiten sintetizar datos realistas a partir de ruido.

Históricamente, los modelos generativos han evolucionado desde enfoques autoregresivos como PixelRNN [1] y modelos de estimación de densidad como NADE [2], hasta modelos más recientes como los autoencoders variacionales (VAE) [3, 4], los modelos adversariales (GANs) [5] y los modelos basados en flujos normalizantes [6, 7].

Entre los distintos enfoques, los modelos basados en difusión han demostrado una gran capacidad para generar imágenes de alta calidad, destacando por su estabilidad durante el entrenamiento frente a otros métodos generativos [8].

2.2. Modelos basados en difusión

Los modelos de difusión consisten en un proceso directo que añade ruido progresivamente a los datos hasta transformarlos en una distribución simple, y un proceso inverso que permite recuperar muestras de la distribución original.

Este enfoque fue formalizado en el marco de modelos generativos basados en score mediante ecuaciones diferenciales estocásticas [8]. En estos modelos, el proceso directo se modela mediante una SDE, mientras que el proceso inverso requiere estimar la función score. Una vez entrenado el modelo, es posible generar nuevas muestras partiendo de ruido gaussiano y aplicando el proceso inverso.

2.3. Procesos estocásticos y ecuaciones diferenciales

Una ecuación diferencial estocástica (SDE) describe la evolución temporal de una variable aleatoria mediante un término determinista y uno estocástico:

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t)dt + g(t)d\mathbf{W}(t).$$

Este tipo de formulación es la base de los modelos de difusión modernos [8]. En este contexto, $\mathbf{x}(t)$ representa una versión ruidosa de los datos. Para $t = 0$ corresponde a los datos originales, mientras que para valores grandes de t se aproxima a una distribución gaussiana.

La simulación de estas ecuaciones se realiza mediante métodos numéricos como Euler-Maruyama, utilizado en este proyecto para la generación de muestras [9].

2.4. Modelos generativos basados en score

Los modelos basados en score estiman el gradiente del logaritmo de la densidad de probabilidad:

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

Este enfoque tiene su origen en el método de *score matching* propuesto por Hyvärinen [10], posteriormente extendido mediante técnicas de *denoising score matching* [11].

Dado que esta función no se conoce explícitamente, se aproxima mediante una red neuronal $\mathbf{s}_\theta(\mathbf{x}, t)$ [12, 13]. Esta red aprende a eliminar el ruido de las muestras en función del tiempo.

En este proyecto, el score se modela mediante una arquitectura tipo U-Net, adecuada para datos de imagen. Una vez entrenado el modelo, el score permite guiar el proceso inverso de difusión y generar nuevas imágenes a partir de ruido.

2.5. Procesos de difusión utilizados

En este proyecto se han considerado dos tipos principales de procesos de difusión, que difieren en la forma en que introducen ruido en los datos: el movimiento browniano y el proceso de Ornstein-Uhlenbeck.

2.5.1. Movimiento Browniano (Variance Exploding)

El modelo basado en movimiento browniano se caracteriza por un incremento progresivo de la varianza a lo largo del tiempo. Este proceso puede describirse mediante la siguiente ecuación diferencial estocástica:

$$d\mathbf{x}(t) = \sigma^t d\mathbf{W}(t),$$

Este tipo de modelos se conocen como *Variance Exploding* (VE) [8].

2.5.2. Proceso de Ornstein-Uhlenbeck (Variance Preserving)

El proceso de Ornstein-Uhlenbeck introduce un término de deriva que empuja las muestras hacia el origen, equilibrando la adición de ruido:

$$d\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)dt + \sqrt{\beta(t)}d\mathbf{W}(t).$$

Estos modelos se conocen como *Variance Preserving* (VP) y constituyen la base de los modelos de difusión discretos como DDPM [14].

2.6. Planificaciones de ruido

La planificación de ruido (*noise schedule*) define cómo evoluciona la intensidad del ruido a lo largo del tiempo en el proceso de difusión.

La elección de esta planificación tiene un impacto directo en la calidad del modelo, como se muestra en trabajos recientes sobre modelos de difusión [14].

2.7. Métodos de generación

Una vez entrenado el modelo para estimar la función score, la generación de nuevas muestras se realiza simulando el proceso inverso de difusión.

2.7.1. Integrador Euler-Maruyama

El método de Euler-Maruyama se utiliza para simular la SDE inversa mediante discretización temporal, siendo una de las técnicas más utilizadas en este contexto [8].

2.7.2. Probability Flow ODE

Alternativamente, el proceso de difusión puede reformularse como una ecuación diferencial ordinaria (ODE), conocida como *probability flow ODE*, que preserva la misma evolución de densidad que la SDE original [8].

2.7.3. Método Predictor-Corrector

El método Predictor-Corrector combina pasos de predicción y corrección basados en dinámicas de tipo Langevin para mejorar la calidad de las muestras generadas [8, 15].

2.8. Métricas de evaluación para modelos generativos

La evaluación de modelos generativos es un aspecto fundamental, ya que no existe una única métrica que capture completamente la calidad de las muestras generadas.

2.8.1. Negative Log-Likelihood

La log-verosimilitud negativa (NLL) mide la capacidad del modelo para asignar alta probabilidad a los datos reales.

2.8.2. Bits per Dimension

El BPD es una transformación de la log-verosimilitud ampliamente utilizada en modelos generativos.

2.8.3. Fréchet Inception Distance

El FID evalúa la similitud entre las distribuciones de las imágenes reales y generadas [16].

2.8.4. Inception Score

El Inception Score mide la calidad y diversidad de las muestras generadas, siendo una métrica común en modelos generativos [5].

Capítulo 3

Diseño del sistema

3.1. Descripción general del sistema

El sistema se organiza en una secuencia de etapas que incluye preparación de datos, difusión, entrenamiento, generación y evaluación. El sistema desarrollado en este proyecto tiene como objetivo la generación de imágenes mediante modelos de difusión basados en ecuaciones diferenciales estocásticas (SDEs). En concreto, se han implementado modelos basados en movimiento browniano (Variance Exploding) y en el proceso de Ornstein-Uhlenbeck (Variance Preserving), así como extensiones para generación condicionada.

El sistema se compone de los siguientes módulos principales:

- Preparación de datos: carga del dataset MNIST y construcción de una versión coloreada para generación condicionada.
- Definición del proceso de difusión: implementación de los procesos forward y reverse.
- Modelo de score: red neuronal encargada de estimar el gradiente del logaritmo de la densidad.
- Entrenamiento: optimización del modelo mediante denoising score matching.
- Generación: obtención de nuevas muestras a partir de ruido.
- Evaluación: comparación de modelos mediante métricas cuantitativas y análisis cualitativo.

3.2. Análisis de requisitos

3.2.1. Requisitos funcionales

El sistema desarrollado debe cumplir los siguientes requisitos funcionales:

- **Generación de imágenes:** El sistema debe ser capaz de generar imágenes sintéticas a partir de ruido gaussiano inicial mediante la simulación del proceso inverso de difusión, utilizando modelos de score entrenados.
- **Generación condicionada:** El sistema debe permitir la generación de imágenes condicionadas tanto por el contenido (dígito) como por atributos visuales (color), integrando dicha información en el modelo generativo.
- **Soporte de distintos procesos de difusión:** El sistema debe implementar y permitir el uso de diferentes formulaciones de procesos de difusión, incluyendo modelos basados en movimiento browniano (Variance Exploding) y procesos de Ornstein-Uhlenbeck (Variance Preserving).
- **Configuración de planificaciones de ruido:** El sistema debe permitir experimentar con distintas estrategias de planificación de ruido (lineal, constante y coseno), facilitando el análisis de su impacto en el rendimiento del modelo.

- **Evaluación de modelos:** El sistema debe proporcionar mecanismos para evaluar la calidad de las imágenes generadas mediante métricas cuantitativas como Fréchet Inception Distance (FID), Inception Score (IS) y Bits per Dimension (BPD).
- **Soporte para distintos métodos de muestreo:** El sistema debe implementar varios métodos de generación de muestras, incluyendo Euler-Maruyama, Predictor-Corrector y formulaciones basadas en ODE.
- **Reconstrucción de imágenes (imputation):** El sistema debe permitir reconstruir imágenes parcialmente observadas mediante técnicas de difusión condicionada.

3.2.2. Requisitos no funcionales

El sistema debe cumplir los siguientes requisitos no funcionales:

- **Entorno de desarrollo:** El sistema debe estar implementado en Python, utilizando la librería PyTorch para la definición y entrenamiento de los modelos.
- **Eficiencia computacional:** El sistema debe ser capaz de ejecutarse de forma eficiente en GPU cuando esté disponible, permitiendo reducir los tiempos de entrenamiento y generación.
- **Modularidad:** El código debe estar organizado en módulos independientes (procesos de difusión, modelos de score, evaluación, etc.), facilitando su mantenimiento, reutilización y extensión.
- **Reproducibilidad:** El sistema debe permitir guardar y cargar modelos entrenados mediante checkpoints, garantizando la reproducibilidad de los experimentos.
- **Escalabilidad:** El sistema debe ser adaptable a distintos conjuntos de datos y configuraciones, permitiendo su aplicación a problemas de mayor complejidad.
- **Legibilidad y documentación:** El código debe seguir buenas prácticas de programación, incluyendo nombres descriptivos, estructura clara y comentarios que faciliten su comprensión.

3.3. Casos de uso

Se han definido los siguientes casos de uso principales, que describen las funcionalidades clave del sistema desarrollado:

3.3.1. Generación de imágenes a partir de ruido

Objetivo: Generar imágenes realistas a partir de ruido gaussiano inicial.

Descripción: El usuario solicita la generación de imágenes sin proporcionar información adicional. El sistema parte de una muestra de ruido y aplica el proceso inverso de difusión utilizando el modelo entrenado.

Interacción: El usuario ejecuta el proceso de muestreo indicando el número de imágenes a generar y el método de sampling. El sistema devuelve un conjunto de imágenes sintéticas.

3.3.2. Generación condicionada por dígito y color

Objetivo: Generar imágenes controlando tanto el contenido (dígito) como la apariencia (color).

Descripción: El usuario especifica un dígito y un color. El modelo condicionado utiliza esta información adicional para guiar el proceso de generación, produciendo imágenes coherentes con las condiciones indicadas.

Interacción: El usuario introduce el dígito y el color deseados. El sistema genera imágenes que respetan ambas condiciones.

3.3.3. Evaluación de modelos generativos

Objetivo: Evaluar la calidad de las imágenes generadas mediante métricas cuantitativas.

Descripción: El sistema compara las imágenes generadas con datos reales utilizando métricas como FID, Inception Score y Bits per Dimension (BPD), proporcionando una medida tanto perceptual como probabilística del rendimiento del modelo.

Interacción: El usuario selecciona el modelo y el conjunto de imágenes generadas. El sistema devuelve los valores de las métricas de evaluación.

3.3.4. Reconstrucción de imágenes incompletas (Imputation)

Objetivo: Reconstruir regiones faltantes de una imagen a partir de la información disponible.

Descripción: A partir de una imagen parcialmente observada, el sistema utiliza el modelo de difusión para inferir las regiones faltantes mediante el proceso inverso condicionado.

Interacción: El usuario proporciona una imagen con una máscara que indica las zonas faltantes. El sistema devuelve una imagen reconstruida.

3.4. Arquitectura del modelo

3.4.1. Modelo ScoreNet basado en U-Net

El modelo utilizado para aproximar la función score es una red neuronal profunda basada en la arquitectura U-Net. La red recibe como entrada una imagen ruidosa x_t junto con el tiempo de difusión t, y produce como salida una estimación del score $\nabla_x \log p_t(x)$.

La arquitectura sigue una estructura *encoder-decoder*. En la parte de codificación (*encoder*), la red reduce progresivamente la resolución espacial de la imagen y aumenta el número de canales, extrayendo características cada vez más abstractas. En la parte de decodificación (*decoder*), la red recupera la resolución original para obtener una salida con el mismo tamaño que la imagen de entrada.

Este diseño resulta especialmente adecuado para el modelado de imágenes, ya que permite reconstruir estructuras complejas a partir de representaciones jerárquicas.

3.4.2. Diagrama de clases del sistema

La Figura 3.1 muestra el diagrama de clases correspondiente a la arquitectura implementada en el proyecto.

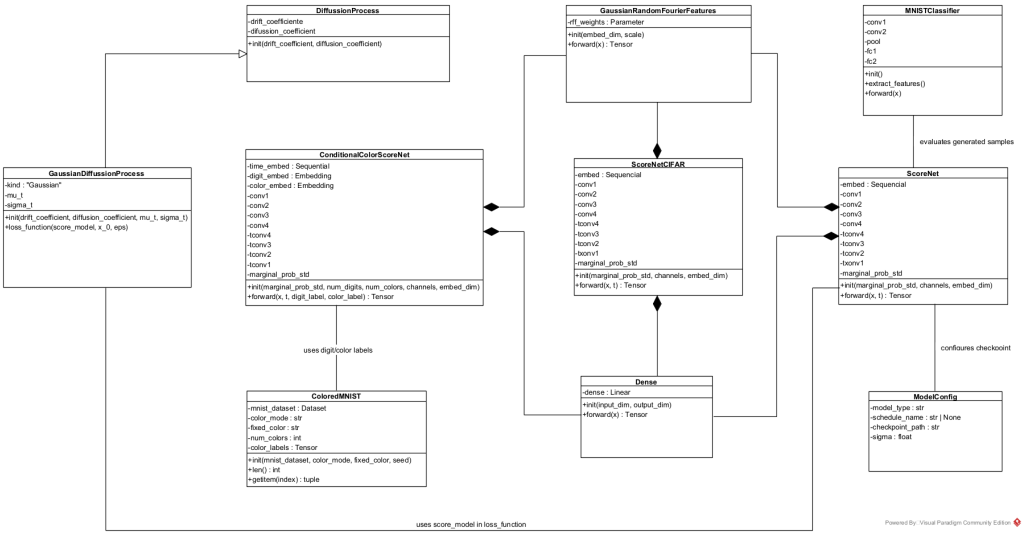

Figura 3.1: Diagrama de clases del sistema de modelos de difusión implementado.

Como se observa en la Figura 3.1, el sistema se organiza en varios componentes principales:

- **Procesos de difusión**: las clases `DiffusionProcess` y `GaussianDiffusionProcess` encapsulan la definición de los procesos forward y las funciones de pérdida utilizadas durante el entrenamiento.
- **Modelos ScoreNet**: las clases `ScoreNet`, `ScoreNetCIFAR` y `ConditionalColorScoreNet` implementan distintas variantes del modelo neuronal encargado de estimar la función score, adaptadas a distintos datasets y configuraciones (MNIST, CIFAR-10 y generación condicionada).
- **Embeddings y componentes auxiliares**: la clase `GaussianRandomFourierFeatures` permite codificar la variable temporal, mientras que módulos como `Dense` se emplean como bloques básicos dentro de la arquitectura.
- **Datos**: la clase `ColoredMNIST` define el dataset extendido con información de color, utilizado para la generación condicionada.
- **Evaluación**: la clase `MNISTClassifier` se emplea para evaluar las muestras generadas mediante métricas como el Inception Score.
- **Configuración**: la clase `ModelConfig` centraliza los parámetros necesarios para definir y cargar los modelos entrenados.

Este diseño modular permite separar claramente las responsabilidades de cada componente, facilitando la extensibilidad del sistema y la experimentación con distintas configuraciones de modelos de difusión.

3.4.3. Codificación temporal mediante Fourier features

El tiempo de difusión t se incorpora al modelo mediante una codificación basada en *Gaussian Random Fourier Features*. Esta técnica transforma el escalar t en un vector de mayor dimensión que facilita el aprendizaje de dependencias no lineales.

Posteriormente, esta representación se proyecta mediante capas lineales y se añade a distintas capas de la red, permitiendo que el modelo adapte su comportamiento en función del nivel de ruido presente en la imagen.

3.5. Validación y pruebas

El sistema se ha validado mediante una combinación de análisis cualitativo y cuantitativo.

Por un lado, se ha evaluado la calidad visual de las imágenes generadas, verificando que el modelo es capaz de reconstruir estructuras coherentes a partir de ruido.

Por otro lado, se han utilizado métricas cuantitativas como el Fréchet Inception Distance (FID) y el Inception Score (IS) para medir la calidad y diversidad de las muestras generadas.

Asimismo, se han realizado comparaciones entre distintas configuraciones del modelo, incluyendo diferentes procesos de difusión, planificaciones de ruido y métodos de muestreo, comprobando que los resultados obtenidos son coherentes con el comportamiento esperado de estos modelos.

3.6. Proceso de entrenamiento

El entrenamiento del modelo se basa en el principio de *denoising score matching*. Para cada imagen del conjunto de entrenamiento, se selecciona un tiempo de difusión t de forma aleatoria y se genera una versión ruidosa x_t añadiendo ruido gaussiano.

El modelo recibe como entrada la imagen x_t y el tiempo t, y se entrena para predecir el ruido añadido. La función de pérdida utilizada es:

$$E\left[\|\sigma_t s_\theta(x_t, t) + z\|^2\right]$$

donde z es ruido gaussiano y σ_t es la desviación estándar del proceso de difusión.

Este procedimiento permite que el modelo aprenda a eliminar el ruido de las imágenes en distintos niveles de perturbación.

El objetivo de este proyecto no ha sido optimizar al máximo el rendimiento de los modelos, sino ilustrar y analizar el comportamiento de los modelos generativos basados en difusión.

Por este motivo, el número de épocas de entrenamiento se ha seleccionado de forma que las muestras generadas sean cualitativamente interpretables (es decir, que los dígitos sean reconocibles), sin centrarse en una optimización exhaustiva de hiperparámetros.

3.7. Proceso de generación de imágenes

Una vez entrenado el modelo, la generación de nuevas imágenes se realiza simulando el proceso inverso de difusión.

El proceso comienza con una muestra inicial de ruido gaussiano y aplica iterativamente el proceso inverso utilizando el score estimado por el modelo. Para ello, se emplea principalmente el método de Euler-Maruyama, que aproxima la solución de la SDE inversa mediante discretización temporal.

A lo largo de este proceso, la imagen evoluciona gradualmente desde ruido hasta adquirir una estructura coherente similar a las imágenes del conjunto de entrenamiento.

3.8. Generación condicionada

Además de la generación no condicionada, se ha implementado un modelo de generación condicionada capaz de controlar características específicas de las imágenes generadas.

El modelo permite generar imágenes correspondientes a un dígito concreto del dataset MNIST, incorporando información de la etiqueta durante el proceso de generación.

Aparte se ha extendido el modelo para generar imágenes en color, incorporando una segunda condición que indica el color deseado. Para ello, se utiliza un dataset MNIST modificado en el que cada imagen se asocia a un color.

En este caso, el modelo aprende una función score condicionada de la forma:

$$s_\theta(x, t, digit, color)$$

Este enfoque permite generar imágenes con combinaciones específicas de contenido (dígito) y apariencia (color), demostrando la capacidad de control del modelo generativo.

3.9. Organización del código

El código del proyecto se organiza en una carpeta `src/`, que contiene los módulos principales del sistema. La estructura separa la definición de procesos de difusión, los modelos neuronales, las utilidades de entrenamiento, la generación condicionada y la evaluación.

Los archivos principales son:

- `diffusion_process.py`: definición general de los procesos de difusión y del integrador Euler-Maruyama.
- `bm_utils.py` y `ou_utils.py`: implementación de los procesos Brownian Motion y Ornstein-Uhlenbeck, incluyendo schedules, sampling y probability flow ODE.
- `score_model.py` y `score_model_cifar.py`: arquitecturas ScoreNet para MNIST y CIFAR-10.
- `conditional_color_score_model.py`, `conditional_color_training_utils.py`: módulos para la generación condicionada por dígito y color.
- `conditional_sampling.py`: funciones para reconstrucción de imágenes mediante *imputation*.
- `evaluation_utils.py`: cálculo de métricas como FID, IS y BPD.
- `diffusion_utilities.py`: funciones auxiliares de visualización.

3.10. Gestión de modelos entrenados y checkpoints

Durante el entrenamiento, los modelos se almacenan en archivos de tipo *checkpoint* que contienen los pesos aprendidos.

En este proyecto, todos los checkpoints se guardan en una carpeta común, utilizando nombres representativos según el tipo de modelo y configuración. Esto facilita su identificación, reutilización y comparación en los experimentos.

Capítulo 4

Desarrollo e implementación

4.1. Metodología de desarrollo

El desarrollo del proyecto se ha realizado de forma incremental, comenzando por la implementación de modelos básicos de difusión y extendiéndose progresivamente hacia configuraciones más complejas.

Se han utilizado notebooks de Jupyter para la experimentación, permitiendo iterar rápidamente sobre distintas configuraciones. Posteriormente, se han organizado los módulos en una estructura más modular para facilitar la reutilización y el mantenimiento del código.

4.2. Planificación del proyecto

El desarrollo del proyecto se ha organizado en distintas fases, incluyendo el estudio teórico, la implementación de los modelos, la experimentación y la evaluación de resultados.

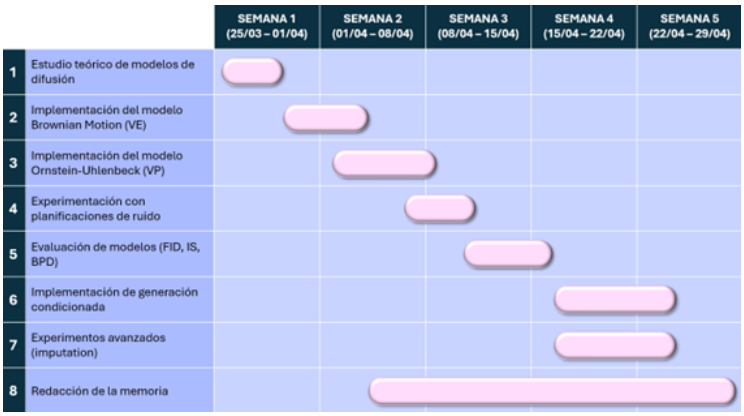

Figura 4.1: Planificación temporal del desarrollo del proyecto.

Como se observa en la Figura 4.1, el proyecto se ha desarrollado de forma progresiva, comenzando con el estudio teórico de los modelos de difusión y continuando con su implementación.

Posteriormente, se han realizado experimentos para analizar el impacto de distintas configuraciones, incluyendo planificaciones de ruido y métodos de muestreo. Finalmente, se han llevado a cabo tareas de evaluación y redacción de la memoria.

4.3. Preparación de los datos

4.3.1. MNIST en escala de grises

Para el entrenamiento de los modelos generativos se ha utilizado el dataset MNIST, compuesto por imágenes en escala de grises de dígitos manuscritos con resolución 28×28.

Las imágenes se normalizan y se transforman en tensores de PyTorch con valores en el rango $[0, 1]$. Este dataset se emplea tanto para la generación no condicionada como para la evaluación de los modelos de difusión.

4.3.2. MNIST coloreado

Para la generación condicionada se ha construido una versión extendida del dataset MNIST en la que cada imagen se transforma en formato RGB mediante la asignación de un color.

Cada imagen en escala de grises se replica en tres canales y se multiplica por un vector RGB que define el color. De este modo, se obtiene un dataset en el que cada muestra está asociada a dos etiquetas: el dígito y el color.

Esta extensión permite entrenar modelos capaces de generar imágenes condicionadas tanto por contenido como por apariencia.

4.4. Implementación de los procesos de difusión

4.4.1. Forward process

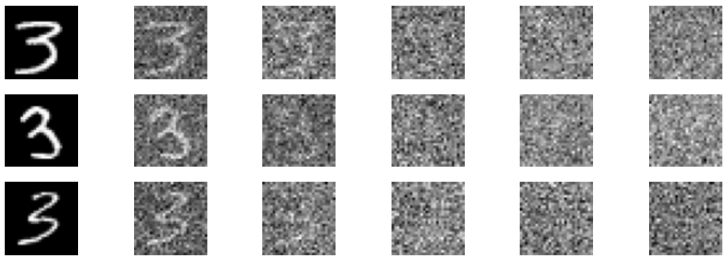

Figura 4.2: Ejemplo del proceso forward de difusión, donde una imagen original se degrada progresivamente mediante la adición de ruido gaussiano.

Como se observa en la Figura 4.2, el proceso directo elimina progresivamente la estructura de la imagen.

El proceso directo de difusión se implementa mediante la adición progresiva de ruido gaussiano a las imágenes originales. Dado un instante de tiempo t, se genera una versión ruidosa x_t a partir de la imagen original x_0 utilizando la media y la varianza definidas por el proceso de difusión.

Este procedimiento permite simular el proceso de degradación de los datos hasta aproximarse a una distribución gaussiana.

4.4.2. Reverse process

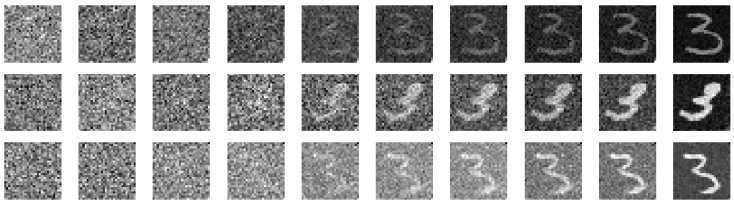

Figura 4.3: Proceso inverso de difusión en el que una muestra inicial de ruido evoluciona hasta generar una imagen coherente mediante el modelo entrenado.

La Figura 4.3 muestra cómo el modelo es capaz de reconstruir la imagen a partir de ruido. El proceso inverso se implementa mediante la simulación de la ecuación diferencial

estocástica inversa. Para ello, se utiliza el score estimado por la red neuronal para guiar la eliminación progresiva del ruido.

Partiendo de una muestra inicial de ruido, se aplican iterativamente actualizaciones que permiten reconstruir la estructura de la imagen.

4.5. Entrenamiento de los modelos

4.5.1. Modelo Brownian Motion

Se ha implementado un modelo basado en movimiento browniano, correspondiente al enfoque *Variance Exploding*. En este caso, la varianza del proceso aumenta con el tiempo, lo que permite explorar regiones amplias del espacio de datos.

El entrenamiento se realiza mediante la función de pérdida de denoising score matching, utilizando muestras perturbadas con distintos niveles de ruido.

4.5.2. Modelo Ornstein-Uhlenbeck

El modelo basado en el proceso de Ornstein-Uhlenbeck corresponde al enfoque *Variance Preserving*. En este caso, la varianza se mantiene acotada, lo que da lugar a una transición más controlada durante el proceso de difusión.

Este modelo se ha utilizado como base principal del proyecto, debido a su estabilidad y mejor rendimiento en la generación de imágenes.

4.5.3. Modelo condicionado para generación controlada

Para la generación condicionada se ha extendido el modelo de score incorporando embeddings para las etiquetas de dígito y color.

4.6. Implementación de los métodos de muestreo

Para la generación de imágenes se han implementado distintos métodos numéricos para simular el proceso inverso:

- Euler-Maruyama: basado en la discretización de la SDE inversa.
- Predictor-Corrector: combina pasos de predicción con refinamientos mediante dinámica de Langevin.
- Probability Flow ODE: versión determinista del proceso de difusión que permite estimar la verosimilitud.

Estos métodos se han comparado experimentalmente para evaluar su impacto en la calidad de las imágenes generadas.

4.7. Validación del funcionamiento

El correcto funcionamiento del sistema se ha validado mediante la visualización del proceso de difusión y generación. En particular, se ha comprobado que:

- El proceso forward degrada progresivamente las imágenes hasta convertirlas en ruido.

- El proceso inverso reconstruye correctamente la estructura de los datos.
- El modelo condicionado permite controlar tanto el dígito como el color de las imágenes generadas.

4.8. Entorno de desarrollo y recursos computacionales

El desarrollo del proyecto se ha realizado en Python utilizando la librería PyTorch para la implementación de los modelos de redes neuronales. Los experimentos se han ejecutado en entornos con soporte para GPU cuando ha sido posible, lo que ha permitido acelerar significativamente el proceso de entrenamiento. Asimismo, se han utilizado notebooks de Jupyter para la organización de los experimentos y la visualización de resultados.

4.9. Consideraciones éticas y sociales

El desarrollo de modelos generativos basados en inteligencia artificial plantea diversas implicaciones éticas y sociales que deben ser consideradas.

En primer lugar, los modelos de difusión son capaces de generar contenido sintético con un alto grado de realismo, lo que abre la puerta a posibles usos indebidos, como la generación de imágenes falsas o manipuladas. Aunque en este proyecto se ha trabajado con datasets sencillos, las mismas técnicas pueden aplicarse a dominios más sensibles, por lo que es importante tener en cuenta el impacto de estas tecnologías.

Por otro lado, el uso de datos como CIFAR-10 puede introducir sesgos, ya que la distribución de las clases y las características visuales no siempre es representativa de la realidad. Haciendo que el modelo pueda aprender y reproducir dichos sesgoss.

Finalmente, este proyecto pone de manifiesto la importancia de combinar el desarrollo técnico con una reflexión ética, especialmente en el contexto de la inteligencia artificial generativa, donde el impacto potencial de estas tecnologías es cada vez mayor.

Capítulo 5

Resultados

5.1. Resultados cualitativos

5.1.1. Generación con Brownian Motion

Se han generado imágenes utilizando el modelo basado en movimiento browniano (Variance Exploding). En este caso, las muestras presentan una mayor dispersión, lo que dificulta la reconstrucción precisa de la estructura original de los datos.

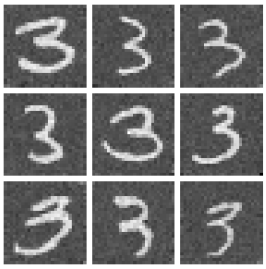

Figura 5.1: Ejemplos de imágenes generadas utilizando el modelo basado en movimiento browniano.

5.1.2. Generación con Ornstein-Uhlenbeck

El modelo basado en el proceso de Ornstein-Uhlenbeck produce imágenes más definidas, gracias a una transición más controlada durante el proceso de difusión.

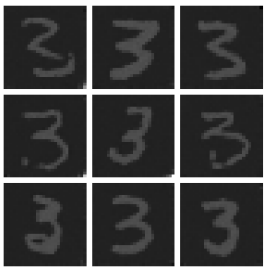

Figura 5.2: Ejemplos de imágenes generadas utilizando el modelo basado en Ornstein-Uhlenbeck.

5.1.3. Generación condicionada

El modelo condicionado permite generar imágenes controlando tanto el dígito como el color. Los resultados muestran que el modelo es capaz de producir combinaciones coherentes entre contenido y apariencia.

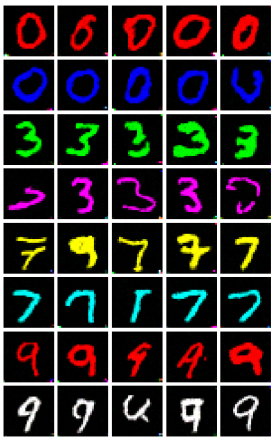

Figura 5.3: Ejemplos de generación condicionada por dígito y color.

5.2. Comparación de planificaciones de ruido

Se han evaluado distintas planificaciones de ruido, incluyendo estrategias lineales y basadas en funciones coseno. Los resultados indican que la planificación coseno proporciona mejores resultados en términos de calidad visual.

Esto se debe a una introducción más progresiva del ruido, lo que facilita el aprendizaje del proceso inverso por parte del modelo.

5.3. Resultados cuantitativos

5.3.1. Comparación de modelos y planificaciones de ruido

Se ha evaluado el impacto del tipo de modelo y la planificación de ruido en la calidad de las imágenes generadas.

Modelo	Schedule	FID	IS	BPD
OU	Cosine	46.98	1.13	-5.34
BM	None	54.26	1.30	-10.50
OU	Linear	121.87	1.52	-0.69
OU	Constant	159.72	1.24	-7.51

Tabla 5.1: Comparación de modelos, planificaciones de ruido y métricas de evaluación.

Los resultados muestran que el modelo basado en Ornstein-Uhlenbeck con planificación coseno obtiene el menor valor de FID, indicando una mejor calidad en las imágenes generadas.

En términos de BPD, el modelo basado en movimiento browniano presenta valores más bajos (mejor ajuste probabilístico), lo que sugiere una mejor aproximación a la distribución de los datos. Sin embargo, esta mejora no se traduce necesariamente en una mayor calidad visual, como reflejan los valores de FID.

Este comportamiento pone de manifiesto la diferencia entre métricas probabilísticas (BPD) y perceptuales (FID), siendo estas últimas más representativas de la calidad visual en tareas de generación de imágenes.

Por otro lado, el Inception Score (IS) debe interpretarse con cautela en este contexto, ya que parte de los experimentos se realizan sobre un único dígito, por lo que la diversidad entre clases no constituye el objetivo principal.

5.3.2. Comparación de métodos de muestreo

Se ha evaluado el impacto de distintos métodos de muestreo en la calidad de las imágenes generadas.

Modelo	Sampler	FID	IS	BPD
OU/VP Cosine	Predictor-Corrector	48.67	1.20	-5.34
OU/VP Cosine	Euler	58.52	1.05	-5.34
BM/VE	Predictor-Corrector	71.62	1.06	-10.50
BM/VE	Euler	92.67	1.33	-10.50
BM/VE	ODE	98.62	1.38	-10.50
OU/VP Cosine	ODE	389.89	1.86	-5.34

Tabla 5.2: Comparación de métodos de muestreo y métricas de evaluación.

Se observa que el método Predictor-Corrector proporciona los mejores resultados en términos de FID para el modelo OU con planificación coseno.

El método Euler-Maruyama presenta un rendimiento ligeramente inferior en términos de calidad visual, aunque mantiene resultados competitivos con menor complejidad computacional.

Por otro lado, el método basado en Probability Flow ODE muestra un comportamiento inestable en este contexto, con valores de FID significativamente mayores en algunas configuraciones.

Cabe destacar que el BPD permanece constante para un mismo modelo independientemente del sampler, ya que esta métrica depende exclusivamente del modelo entrenado y no del procedimiento de generación.

En conjunto, estos resultados indican que la elección del método de muestreo influye principalmente en la calidad perceptual de las muestras generadas, mientras que la estimación probabilística del modelo permanece inalterada.

5.3.3. Evolución del modelo durante el entrenamiento

Se ha analizado la evolución del modelo a lo largo del entrenamiento mediante métricas cuantitativas como el Fréchet Inception Distance (FID), el Inception Score (IS) y los Bits per Dimension (BPD).

Epoch	FID	IS	BPD
10	1404.11	1.99	6.40
20	231.98	1.72	2.88
50	65.96	1.22	0.07
100	90.83	1.46	-4.15
200	52.50	1.09	-7.00

Tabla 5.3: Evolución de las métricas FID, IS y BPD durante el entrenamiento.

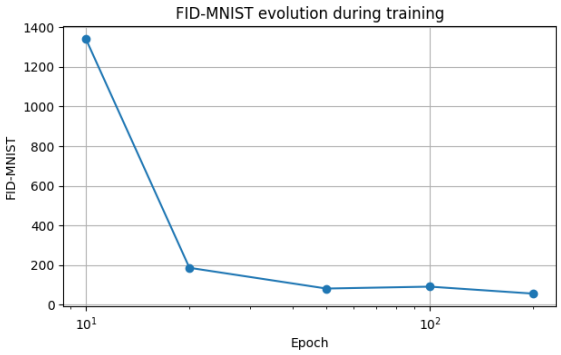

Figura 5.4: Evolución del FID durante el entrenamiento del modelo.

Como se observa en la Figura 5.4 y en la Tabla 5.3, el modelo mejora significativamente a lo largo del entrenamiento.

El FID disminuye de forma drástica en las primeras épocas, indicando una mejora rápida en la calidad visual de las imágenes generadas. Posteriormente, la mejora es más progresiva, mostrando una tendencia hacia la convergencia.

Por otro lado, el BPD presenta una disminución continua, pasando de valores positivos a valores negativos. Esto indica una mejora progresiva en la aproximación de la distribución de los datos, incluso en etapas donde el FID presenta pequeñas fluctuaciones.

Este comportamiento pone de manifiesto que las métricas perceptuales (FID) y probabilísticas (BPD) capturan aspectos diferentes del modelo: mientras que el FID refleja la calidad visual de las muestras, el BPD evalúa el ajuste del modelo a la distribución subyacente de los datos.

El Inception Score presenta variaciones más suaves, lo que sugiere que la diversidad de las muestras se mantiene relativamente estable durante el entrenamiento.

5.4. Discusión de resultados

Los resultados obtenidos muestran que el modelo basado en Ornstein-Uhlenbeck combinado con una planificación de ruido tipo coseno proporciona el mejor rendimiento.

Aunque Euler-Maruyama presenta menor calidad que Predictor-Corrector, su simplicidad y menor coste computacional lo hacen una alternativa práctica.

La generación condicionada confirma la capacidad del modelo para aprender representaciones ricas de los datos, permitiendo controlar características específicas como el dígito y el color.

En conjunto, estos resultados validan la eficacia de los modelos de difusión para la generación de imágenes de alta calidad.

5.5. Reconstrucción de imágenes (Imputation)

Se ha evaluado de forma exploratoria la capacidad del modelo para reconstruir imágenes parcialmente observadas mediante técnicas de *imputation*.

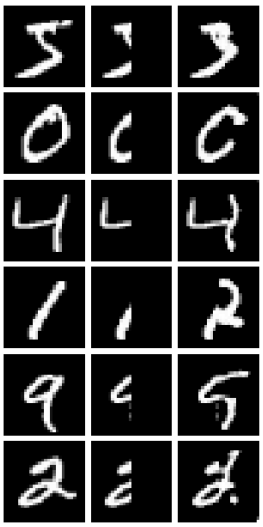

Figura 5.5: Ejemplo de reconstrucción de imágenes. En cada fila se muestra la imagen original (izquierda), la imagen reconstruida (centro) y la versión parcialmente observada (derecha).

Como se observa en la Figura 5.5, el modelo es capaz de reconstruir imágenes incluso cuando algunas regiones están parcialmente ocultas o contienen información incompleta. Sin embargo, la calidad de la reconstrucción disminuye especialmente en las zonas donde la información disponible es más limitada. Estos resultados muestran que el modelo tiene capacidad para realizar tareas de reconstrucción a partir de datos incompletos, aunque también ponen de manifiesto ciertas limitaciones en escenarios con alta incertidumbre o pérdida significativa de información.

Capítulo 6

Conclusiones

6.1. Conclusiones principales

En este proyecto se ha abordado el estudio e implementación de modelos generativos basados en procesos de difusión, analizando tanto su formulación teórica como su comportamiento práctico en tareas de generación de imágenes.

En primer lugar, se han implementado modelos basados en movimiento browniano (Variance Exploding) y en procesos de Ornstein-Uhlenbeck (Variance Preserving), observándose diferencias significativas en su comportamiento. En particular, el modelo basado en Ornstein-Uhlenbeck ha mostrado un mejor rendimiento en términos de calidad visual de las muestras generadas, especialmente cuando se combina con una planificación de ruido tipo coseno.

Asimismo, se ha comprobado que la elección de la planificación de ruido tiene un impacto notable en el rendimiento del modelo. Las estrategias basadas en funciones suaves, como la planificación coseno, permiten una transición más progresiva en el proceso de difusión, lo que facilita el aprendizaje del proceso inverso y se traduce en mejores resultados en términos de FID.

Por otro lado, el análisis de distintos métodos de muestreo ha puesto de manifiesto que técnicas como Predictor-Corrector ofrecen un mejor equilibrio entre calidad y estabilidad, mientras que métodos más simples como Euler-Maruyama, aunque menos precisos, presentan una menor complejidad computacional y resultados razonables. En contraste, el uso de formulaciones deterministas basadas en ODE ha mostrado un comportamiento menos robusto en este contexto.

El estudio de métricas de evaluación ha permitido evidenciar la diferencia entre métricas probabilísticas y perceptuales. Mientras que el BPD proporciona información sobre la calidad del ajuste del modelo a la distribución de los datos, métricas como FID resultan más adecuadas para evaluar la calidad visual de las imágenes generadas. Este hecho pone de relieve la importancia de utilizar múltiples métricas de forma complementaria.

Además, se ha explorado la capacidad del modelo para realizar generación condicionada, demostrando que es posible controlar características específicas de las imágenes generadas, como el dígito y el color. Este resultado confirma que los modelos de difusión son capaces de aprender representaciones ricas y manipulables del espacio de datos.

Finalmente, los experimentos adicionales realizados sobre el conjunto CIFAR-10 muestran que, aunque el modelo es capaz de capturar estructuras globales en datos más complejos, la calidad de las muestras se ve limitada, lo que indica la necesidad de arquitecturas más avanzadas o un entrenamiento más intensivo para este tipo de problemas.

6.2. Limitaciones del trabajo

A pesar de los resultados obtenidos, este trabajo presenta diversas limitaciones. En primer lugar, el entrenamiento de los modelos se ha realizado con recursos computacionales limitados, lo que restringe el número de épocas y la complejidad de las arquitecturas utilizadas. Esto afecta directamente a la calidad de las muestras generadas, especialmente en datasets más complejos como CIFAR-10.

Por otro lado, el uso de conjuntos de datos simplificados, como MNIST o subconjuntos

de una sola clase, limita la capacidad de evaluar completamente el rendimiento del modelo en escenarios más realistas y diversos.

Asimismo, la estimación de métricas probabilísticas como el BPD depende de aproximaciones numéricas que pueden introducir errores, especialmente en el caso de métodos basados en ODE.

Finalmente, algunos experimentos, como la reconstrucción de imágenes (imputation), se han abordado de forma exploratoria, sin una optimización específica del modelo para este tipo de tareas.

6.3. Líneas futuras

Como posibles líneas de trabajo futuro, se plantea en primer lugar la mejora de la calidad de los modelos mediante el uso de arquitecturas más avanzadas y un mayor número de épocas de entrenamiento.

Asimismo, sería interesante extender los experimentos a conjuntos de datos más complejos y de mayor resolución, así como explorar técnicas de conditioning más sofisticadas, como el uso de embeddings semánticos.

Otra línea de interés consiste en mejorar los métodos de muestreo, evaluando técnicas más eficientes que permitan reducir el número de pasos necesarios sin comprometer la calidad de las muestras.

Finalmente, se propone profundizar en el uso de modelos de difusión para tareas adicionales, como la reconstrucción de imágenes, la super-resolución o la edición de imágenes, donde este tipo de modelos ha demostrado un gran potencial en trabajos recientes.

Bibliografía

[1] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu, "Pixel recurrent neural networks," in *International Conference on Machine Learning*, vol. 48, pp. 1747–1756, 2016.

[2] B. Uria, M.-A. Côté, K. Gregor, I. Murray, and H. Larochelle, "Neural autoregressive distribution estimation," *Journal of Machine Learning Research*, vol. 17, no. 205, pp. 1–37, 2016.

[3] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *International Conference on Learning Representations*, 2014.

[4] D. Rezende, S. Mohamed, and D. Wierstra, "Stochastic backpropagation and approximate inference in deep generative models," in *International Conference on Machine Learning*, pp. 1278–1286, 2014.

[5] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in Neural Information Processing Systems*, pp. 2672–2680, 2014.

[6] D. J. Rezende and S. Mohamed, "Variational inference with normalizing flows," in *International Conference on Machine Learning*, pp. 1530–1538, 2015.

[7] L. Dinh, J. Sohl-Dickstein, and S. Bengio, "Density estimation using real nvp," in *International Conference on Learning Representations*, 2017.

[8] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole, "Score-based generative modeling through stochastic differential equations," in *International Conference on Learning Representations*, 2021.

[9] A. Suárez, "Stochastic differential equations; generative ai for images; diffusion models for generative ai," 2024. Material docente interno, Universidad Autónoma de Madrid (UAM).

[10] A. Hyvärinen, "Estimation of non-normalized statistical models by score matching," *Journal of Machine Learning Research*, 2005.

[11] P. Vincent, "A connection between score matching and denoising autoencoders," *Neural Computation*, 2011.

[12] Y. Song and S. Ermon, "Generative modeling by estimating gradients of the data distribution," in *NeurIPS*, 2019.

[13] Y. Song and S. Ermon, "Improved techniques for training score-based generative models," in *NeurIPS*, 2020.

[14] A. Q. Nichol and P. Dhariwal, "Improved denoising diffusion probabilistic models," in *International Conference on Machine Learning*, pp. 8162–8171, 2021.

[15] T. Dockhorn, A. Vahdat, and K. Kreis, "Score-based generative modeling with critically-damped langevin diffusion," *ICLR*, 2022.

[16] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "Gans trained by a two time-scale update rule converge to a local nash equilibrium," in *NeurIPS*, 2017.

[17] Z. Guo, J. Lang, S. Huang, Y. Gao, and X. Ding, "A comprehensive review on noise control of diffusion model," *arXiv preprint*, 2025.

[18] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[19] Y. Song and D. P. Kingma, "How to train your energy-based models," *arXiv preprint*, 2021.

[20] S. Mohamed and B. Lakshminarayanan, "Learning in implicit generative models," *arXiv preprint*, 2016.

[21] D. Tran, R. Ranganath, and D. Blei, "Deep and hierarchical implicit models," *arXiv preprint*, 2017.

[22] Y. Song, S. Garg, J. Shi, and S. Ermon, "Sliced score matching," in *UAI*, 2019.

Capítulo A

Material adicional

En este capítulo se recogen materiales complementarios al desarrollo principal del proyecto. Por un lado, se incluyen derivaciones matemáticas detalladas de los modelos de difusión, con el objetivo de justificar rigurosamente las ecuaciones empleadas.

Por otro lado, se presentan resultados experimentales adicionales que complementan los mostrados en el cuerpo principal del trabajo.

A.1. Derivación de las ecuaciones diferenciales estocásticas

Un proceso de difusión continuo se modela mediante una ecuación diferencial estocástica (SDE) en el sentido de Itô:

$$d\mathbf{x}(t) = f(\mathbf{x}(t), t)\, dt + g(t)\, d\mathbf{W}(t) \tag{A.1}$$

donde $\mathbf{W}(t)$ es un proceso de Wiener (movimiento browniano), que satisface:

$$E[d\mathbf{W}(t)] = 0, \quad E[d\mathbf{W}(t) d\mathbf{W}(t)^T] = I\, dt \tag{A.2}$$

Esta ecuación describe la evolución de una variable aleatoria en el tiempo bajo dos efectos:

- Un término determinista (drift): $f(\mathbf{x}, t)$
- Un término estocástico (difusión): $g(t)$

A.1.1. Ecuación de Fokker-Planck

La evolución de la densidad de probabilidad $p_t(\mathbf{x})$ asociada a la SDE viene dada por la ecuación de Fokker-Planck:

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\nabla_{\mathbf{x}} \cdot (f(\mathbf{x}, t) p_t(\mathbf{x})) + \frac{1}{2} \nabla_{\mathbf{x}} \nabla_{\mathbf{x}}^T : \Big(g^2(t) I\, p_t(\mathbf{x}) \Big) \tag{A.3}$$

Esta ecuación describe cómo evoluciona la distribución de probabilidad en el tiempo :contentReference[oaicite:0]index=0.

A.2. Proceso directo (forward diffusion)

El objetivo del proceso forward es transformar progresivamente la distribución de datos $p_0(\mathbf{x})$ en una distribución simple (normal estándar).

A.2.1. Caso Brownian Motion (Variance Exploding)

Se define la SDE:

$$d\mathbf{x}(t) = g(t)\, d\mathbf{W}(t) \tag{A.4}$$

Esta ecuación no tiene término de drift, por lo que la solución es:

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t g(s)\, d\mathbf{W}(s) \tag{A.5}$$

Usando propiedades del movimiento browniano, esta integral sigue una distribución normal:

$$\mathbf{x}(t) \sim \mathcal{N}\left(\mathbf{x}(0), \sigma_t^2 I\right) \tag{A.6}$$

donde:

$$\sigma_t^2 = \int_0^t g^2(s)\, ds \tag{A.7}$$

Esto justifica la expresión utilizada en el código para generar muestras ruidosas.

A.2.2. Caso Ornstein-Uhlenbeck (Variance Preserving)

Se define la SDE:

$$d\mathbf{x}(t) = -\frac{1}{2}\beta(t)\mathbf{x}(t)\, dt + \sqrt{\beta(t)}\, d\mathbf{W}(t) \tag{A.8}$$

Esta es una ecuación lineal cuya solución se obtiene mediante factor integrante:

$$\mathbf{x}(t) = \mu_t \mathbf{x}(0) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, I) \tag{A.9}$$

con:

$$\mu_t = \exp\left(-\frac{1}{2}\int_0^t \beta(s)\, ds\right) \tag{A.10}$$

$$\sigma_t^2 = 1 - \mu_t^2 \tag{A.11}$$

Este proceso preserva la varianza en el límite, convergiendo a una normal estándar.

A.3. Derivación del proceso inverso

El resultado clave de los modelos de difusión es que el proceso inverso también es una SDE.

A partir de la ecuación de Fokker-Planck y aplicando el teorema de Anderson (1982), se obtiene:

$$d\mathbf{x}(t) = \left[f(\mathbf{x}(t), t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})\right] dt + g(t) d\mathbf{W}(t) \tag{A.12}$$

Este resultado es fundamental: permite invertir el proceso de difusión si conocemos el score:

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \tag{A.13}$$

Sin embargo, esta cantidad es desconocida y debe ser aproximada con una red neuronal.

A.4. Estimación del score mediante redes neuronales

Se define el score como:

$$s_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \tag{A.14}$$

A.4.1. Derivación de la función de pérdida

El objetivo ideal sería minimizar:

$$E\left[\|s_\theta(\mathbf{x}, t) - \nabla_{\mathbf{x}} \log p_t(\mathbf{x})\|^2\right] \tag{A.15}$$

Pero el término $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ es desconocido.
Usando el truco de denoising score matching, si:

$$\mathbf{x}(t) = \mathbf{x}(0) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, I) \tag{A.16}$$

entonces:

$$\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) = -\frac{\mathbf{z}}{\sigma_t} \tag{A.17}$$

Por tanto, la función de pérdida se convierte en:

$$\mathcal{L}(\theta) = E\left[\left\|s_\theta(\mathbf{x}(t), t) + \frac{\mathbf{z}}{\sigma_t}\right\|^2\right] \tag{A.18}$$

Esta es exactamente la loss implementada en el proyecto.

A.5. Discretización: método de Euler-Maruyama

Para simular la SDE inversa se utiliza el esquema de Euler-Maruyama:

$$\mathbf{x}_{t-1} = \mathbf{x}_t + f(\mathbf{x}_t, t)\Delta t - g^2(t) s_\theta(\mathbf{x}_t, t)\Delta t + g(t)\sqrt{\Delta t}\mathbf{z} \tag{A.19}$$

Este método es una generalización del método de Euler para ecuaciones diferenciales ordinarias.

A.6. Probability Flow ODE

Existe una formulación determinista equivalente al proceso inverso:

$$d\mathbf{x}(t) = \left[f(\mathbf{x}, t) - \frac{1}{2} g^2(t) s_\theta(\mathbf{x}, t)\right] dt \tag{A.20}$$

Esta ecuación permite generar muestras sin ruido, y además calcular la verosimilitud mediante el cambio de variables.

A.7. Interpretación intuitiva

El modelo de difusión puede entenderse como:

- **Forward**: destrucción progresiva de la información (ruido)
- **Score**: aprender la dirección de máxima probabilidad
- **Reverse**: reconstrucción paso a paso eliminando ruido

En términos físicos, el proceso es análogo a una partícula en un campo de potencial, donde el score actúa como una fuerza que guía hacia regiones de alta probabilidad.

A.8. Resultados adicionales

A.8.1. Reconstrucción de imágenes (imputation)

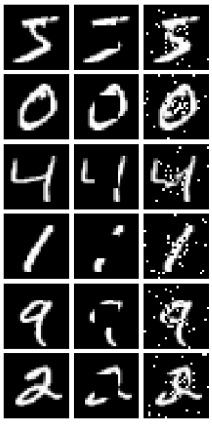

Figura A.1: Reconstrucción de imágenes con región central faltante.

A.8.2. Generación adicional

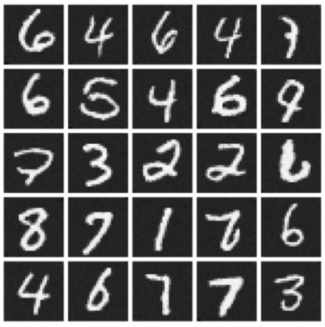

Figura A.2: Generación adicional con modelo OU.

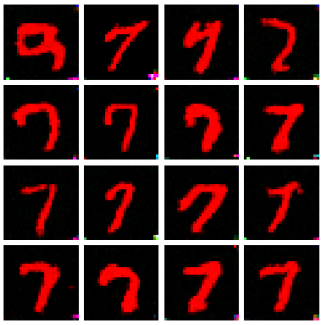

Figura A.3: Generación condicionada adicional.

A.8.3. Resultados en CIFAR-10

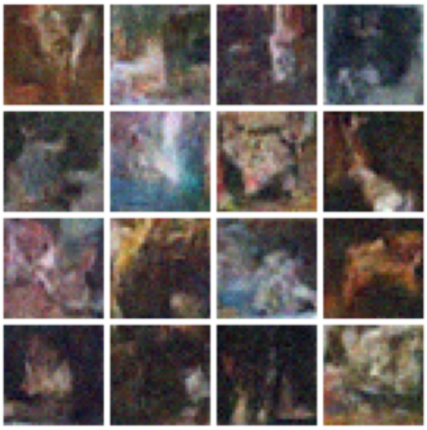

Figura A.4: Generación de imágenes de gatos en CIFAR-10.